

Google Colab Pro for LoRA Fine-Tuning — Feasibility & Limitations

Colab Pro can train a per-character Flux LoRA and save us the fal.ai training fee. But Colab is a *session*, not a *host* — it cannot stand up an inference endpoint for our CLI pipeline to call. The clean split is **train on Colab, run inference on fal.ai** using the LoRA files Colab produces.

Where this fits in our pipeline

- **v2.0 (compositing engine)** is our current shipped consistency solution: reuse the same character sprite PNGs, so characters are pixel-identical but read slightly “pasted.”
 - **pipeline/lora.py** is the next bet: a **per-character Flux LoRA** so the model actually *learns* each character and paints them naturally into any pose — “consistent AND naturally painted.” It is currently wired to **fal.ai** (flux-lora-fast-training, needs FAL_KEY, paid). Using Colab would replace only the **training** step.
-

The caveats that matter

1. VRAM is the binding constraint

Flux.1-dev is a 12B model. Comfortable LoRA training wants **~24 GB VRAM**.

- Colab Pro gives you whatever GPU is free — usually a **T4 (16 GB)**, sometimes an **L4 (24 GB)** or **A100 (40 GB)**. You **cannot reliably pick**.
- On a **16 GB T4** you *must* use a memory-optimized trainer (ostris `ai-toolkit`, or kohya / `sd-scripts`) with fp8/quantization + gradient checkpointing. It works, but a 1000-step LoRA is roughly **1–2+ hours**.
- On an **A100** it’s comfortable and fast — roughly **20–40 minutes**.

2. Colab is a session, not a host

- Sessions **time out** (idle disconnect; ~24 h ceiling even when active) and the VM is **ephemeral** — everything is wiped on disconnect. we must save the `.safetensors` LoRA out to **Google Drive or Hugging Face immediately**.
- we **cannot run inference-as-a-service** from Colab for our book pipeline. Colab is for *producing the LoRA weight files*, not for standing up an endpoint our local CLI (`lora.py`) can call.

3. Inference still needs a home

Training on Colab yields 5 `.safetensors` files (one per character). Inference (`fal-ai/flux-lora` in our code) still has to run **somewhere**:

- **Keep fal.ai for inference only** — upload the Colab-trained LoRAs. fal's `flux-lora` accepts an arbitrary LoRA path, so our existing `generate()` works with almost no change. (*Recommended.*)
- Or run inference in Colab batch too — but then Colab becomes our whole render farm: session-limited, and undrivable from the local CLI pipeline.

4. Training-set quality decides everything

LoRA quality lives or dies on the input images. Before spending GPU hours we should confirm our atlas + reference sheets (`output/assets/<char>/*.raw.png`, `output/refs/<char>/`) give **enough images and enough pose/angle variety** per character. Too few / too similar → the LoRA overfits and only reproduces one pose.

Recommended architecture

Step	Where	Why
Train LoRA (per character)	Colab Pro (ai-toolkit, T4/A100-aware)	Saves the fal.ai training fee
Store weights	Google Drive / Hugging Face	Colab VM is ephemeral
Run inference	fal.ai flux-lora	Persistent, callable from local CLI; minimal code change

This keeps `lora.py`'s inference path intact while cutting the training cost, and avoids trying to make Colab a persistent host (which it is not).

Proposed next steps

1. **Audit the training set** — count images & pose variety per character; flag anyone too thin to train well.
2. **Colab training notebook** — ai-toolkit, T4/A100-aware, reads our `output/assets/<char>/*.raw.png` + `output/refs`, writes per-character LoRAs to Drive/HF.
3. **Refactor `lora.py`** — let `train()` consume **externally-trained LoRA URLs** (skip fal training, keep fal inference).